

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

21 - Pseudoinstructions and Decoding

Introduction

- MIPS jump instruction has the simplest addressing
 - It uses the final MIPS instruction format, called the *J-type*
- Branch instruction
 - Must specify branch address

Pseudoinstructions

- Assembly language is an interface to higher-level SW
- Assembler can treat common variations of machine language instructions as if they were instructions in their own right
- HW need not implement these instructions; however, their appearance in assembly language simplifies translation and programming
- Such instructions are called **pseudoinstructions**

Pseudoinstructions

- Register \$zero is used to create assembly language instruction that copies contents of one register to another
- MIPS assembler accepts this instruction even though it is not found in MIPS architecture:

`move $t0,$t1 # register $t0 gets register $t1`

Pseudoinstructions

- Register \$zero is used to create assembly language instruction that copies contents of one register to another
- MIPS assembler accepts this instruction even though it is not found in MIPS architecture:

`move $t0,$t1 # register $t0 gets register $t1`

- Assembler converts this assembly language instruction into the machine language equivalent of the following instruction:

`add $t0,$zero,$t1 # $t0 gets 0 + $t1`

Pseudoinstructions

- MIPS assembler also converts `b<1>t` (branch on less than) into two instructions `s<1>t` and `bne`
- Other examples include `bgt`, `bge`, and `b<1>e`
- Pseudoinstructions give a richer set of assembly language instructions than those implemented by HW
- To get best performance, however, use real MIPS instructions

Decoding Machine Language

- Sometimes you are forced to reverse-engineer machine language to create the original assembly language
- One example is when looking at “core dump”
- Figures shows encoding of the fields for the MIPS machine language
 - Helps when translating by hand between assembly language and machine language

MIPS Instruction Encoding

op(31:26)								
28-26	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
31-29								
0(000)	R-format	Bltz/gez	jump	jump & link	branch eq	branch ne	blez	bgtz
1(001)	add immediate	addiu	set less than imm.	set less than imm. unsigned	andi	ori	xori	load upper immediate
2(010)	TLB	FlPt						
3(011)								
4(100)	load byte	load half	lwl	load word	load byte unsigned	load half unsigned	lwr	
5(101)	store byte	store half	swl	store word			swr	
6(110)	load linked word	lwcl						
7(111)	store cond. word	swcl						

MIPS Instruction Encoding

op(31:26)=000000 (R-format), funct(5:0)								
2-0 5-3	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	shift left logical		shift right logical	sra	sllv		srlv	srav
1(001)	jump register	jalr			syscall	break		
2(010)	mfhi	mthi	mflo	mtlo				
3(011)	mult	multu	div	divu				
4(100)	add	addu	subtract	subu	and	or	xor	not or (nor)
5(101)			set l.t.	set l.t. unsigned				
6(110)								
7(111)								

MIPS Instruction Encoding

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

Example

- What is the assembly language statement corresponding to this machine instruction?

00af8020_{hex}

Example (cont.)

- First step in converting hexadecimal to binary is to find the op fields:

```
(Bits: 31 28 26                                5  2 0)
      0000 0000 1010 1111 1000 0000 0010 0000
```

- Look at op field to determine operation
- Referring to the previous figures
 - When bits 31–29 are 000 and bits 28–26 are 000, it is R-format instruction

Example (cont.)

- Let's reformat the binary instruction into R-format fields

op	rs	rt	rd	shamt	funct
000000	00101	01111	10000	00000	100000

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

Example (cont.)

- To determine the operation of an R-format instruction
 - In this case, bits 5–3 are 100 and bits 2–0 are 000

```

op      rs      rt      rd      shamt  funct
000000  00101   01111   10000  00000  100000
    
```

op(31:26)=000000 (R-format), funct(5:0)								
2-0 5-3	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	shift left logical		shift right logical	sra	sllv		srlv	srav
1(001)	jump register	jalr			syscall	break		
2(010)	mfhi	mthi	mflo	mtlo				
3(011)	mult	multu	div	divu				
4(100)	add	addu	subtract	subu	and	or	xor	not or (nor)
5(101)			set l.t.	set l.t. unsigned				
6(110)								
7(111)								

Example (cont.)

- To determine the operation of an R-format instruction
 - In this case, bits 5–3 are 100 and bits 2–0 are 000
 - which means this binary pattern represents an **add** instruction

op(31:26)=000000 (R-format), funct(5:0)								
2–0 5–3	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	shift left logical		shift right logical	sra	sllv		srlv	srav
1(001)	jump register	jalr			syscall	break		
2(010)	mfhi	mthi	mflo	mtlo				
3(011)	mult	multu	div	divu				
4(100)	add	addu	subtract	subu	and	or	xor	not or (nor)
5(101)			set l.t.	set l.t. unsigned				
6(110)								
7(111)								

Example (cont.)

- Decode rest of instruction by looking at field values
 - Decimal values are 5 for rs field, 15 for rt, 16 for rd (shamt is unused)

op	rs	rt	rd	shamt	funct
000000	00101	01111	10000	00000	100000

Name	Register number	Usage	Preserved on call?
\$zero	0	The constant value 0	n.a.
\$v0-\$v1	2-3	Values for results and expression evaluation	no
\$a0-\$a3	4-7	Arguments	no
\$t0-\$t7	8-15	Temporaries	no
\$s0-\$s7	16-23	Saved	yes
\$t8-\$t9	24-25	More temporaries	no
\$gp	28	Global pointer	yes
\$sp	29	Stack pointer	yes
\$fp	30	Frame pointer	yes
\$ra	31	Return address	yes

Example (cont.)

- Decode rest of instruction by looking at field values
 - Decimal values are 5 for rs field, 15 for rt, 16 for rd (shamt is unused)
 - These numbers represent registers \$a1, \$t7, and \$s0.

Name	Register number	Usage	Preserved on call?
\$zero	0	The constant value 0	n.a.
\$v0-\$v1	2-3	Values for results and expression evaluation	no
\$a0-\$a3	4-7	Arguments	no
\$t0-\$t7	8-15	Temporaries	no
\$s0-\$s7	16-23	Saved	yes
\$t8-\$t9	24-25	More temporaries	no
\$gp	28	Global pointer	yes
\$sp	29	Stack pointer	yes
\$fp	30	Frame pointer	yes
\$ra	31	Return address	yes

Example (cont.)

- Now we can reveal the assembly instruction:

```
add $s0,$a1,$t7
```